

AlpacaAgent

The operator's handbook

A practical, detailed guide to setup, research, shadow testing, controlled paper operation, and daily review.

DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

1. The most effective way to use the system	3
2. Understand the roles and boundaries	4
3. Set up a clean Windows session	5
4. Open the dashboard in the right mode	6
5. Research before enabling anything	7
6. Run a genuine shadow-testing phase	8
7. Know exactly what can become an order	9
8. Conduct a controlled paper pilot	10
9. Understand how exits really work	11
10. Build an efficient daily and weekly routine	12
11. Limits, evidence, and the next learning cycle	13

1. The most effective way to use the system

AlpacaAgent works best as an evidence-gathering paper-trading laboratory. Use it to learn whether a repeatable setup survives data checks, risk rules, order handling, and actual paper fills. The immediate objective is a process you can explain and reproduce: what the system saw, why it acted or refused, what happened at the broker, and what you learned.

This guide describes the local AlpacaAgent repository, not every feature offered by the Alpaca brokerage platform. It was checked against source revision 235b087 on September 18, 2026. Local documentation contains older milestones and some contradictory examples; the implementation takes precedence here. No account was queried and no trading controls were changed while preparing these guides.

Your recommended progression

1. Learn the dashboard and read existing reports without making orders possible.
2. Reproduce underlying-price backtests from saved bars. Write a research hypothesis before looking for the best result.
3. Run broker diagnostics and standalone shadow scans. Record missing data and failures separately from valid scans with no ideas.
4. Review one executable playbook and a narrow symbol universe. Approve only after you can explain its evidence and limitations.
5. Run a supervised paper pilot with unchanged limits. Review order outcomes and exit behavior, not just profit.
6. Expand only after resolving operational defects and evaluating enough observations. There is no live-trading switch in this project.

Start with the fewest moving parts that answer your question. A historical-research session needs neither a running controller nor an armed control file. A dashboard-reading session does not need broker credentials.

How to read this guide set

Volume 1 explains the operating workflow in order. Volume 2 supplies worked use cases, agent prompts, and research templates. Volume 3 is the desk reference for commands, failures, recovery, and the meaning of records. Read Volume 1 first; keep Volume 3 nearby when running the system.

What a successful first week looks like

You can open the dashboard, identify the timestamp of its evidence, reproduce a report, distinguish a scanner failure from no signal, explain every readiness gate, and locate the corresponding cycle or journal record. More trades are not a success criterion. A refusal that protects an invariant is useful evidence.

Do not treat the historical account and subscription descriptions in `HANDOVER.md` as a current account check. They are notes from an earlier session. The same applies to references to previously exposed credentials: if those credentials have not already been rotated, rotate them before reuse.

2. Understand the roles and boundaries

Star: idea generation

Star's scanner combines completed daily bars, deterministic signals, and eligible option contracts. It can produce shadow ideas, enabled proposals, and reasons for skipping candidates. It does not own the broker connection and does not decide how much risk the account may take.

The normal CLI universe is SPY, QQQ, and IWM. DIA is also accepted through --symbols. These are the supported ETF inputs for this workflow; the fact that a chat tool can describe another stock does not add that stock to the scanner.

Moon: deterministic rules

Moon evaluates a fully specified idea against a fresh, reconciled risk state. It recomputes important quantities rather than accepting a persuasive thesis. A chat conversation with Moon explains rules; it does not modify the deterministic engine or make an exception.

Houston: execution and reconciliation

Houston's executor compares broker information with the local activity history, fill ledger, and order journal. Orders pass through a durable reserve, claim, and submit sequence. The broker transport makes one submission attempt per claimed authorization. An uncertain outcome remains an incident for reconciliation, not permission to retry.

Astra and Nova: interpretation

Astra explains workspace status and saved records. Nova is available in generative mode as a general assistant. The agent names are different conversational roles, not five independent trading accounts or five risk budgets. All trading limits are shared.

The controller coordinates the work

One controller cycle reconciles first, resolves eligible terminal intents, checks exits, then considers entries. If reconciliation fails, the cycle does not continue into ordinary trading. The runtime lock prevents two controllers from owning the same runtime directory.

The dashboard is a view of records. Opening it does not start the controller, and opening a served workspace without --every does not start periodic trading cycles. The machine, terminal process, credentials, network, and applicable data feeds must be available for scheduled operation.

Capabilities you should not assume

The current executor supports long single-leg calls and puts. Some debit spreads can be constructed and validated earlier in the pipeline, but multi-leg execution is refused. Chat cannot place or cancel orders, arm controls, approve playbooks, release intents, or edit limits. There is no cancel/replace workflow, automatic order chasing, complete assignment/exercise handling, or options-level historical P&L backtest.

3. Set up a clean Windows session

Establish the working directory and Python

Run the examples from the repository root in PowerShell. Python 3.11 or newer is required. In this documentation-building environment, a plain python command was not on PATH; your normal terminal may differ. Check before diagnosing the application itself.

```
Set-Location 'C:\Users\eveli\OneDrive\Desktop\AlpacaAgent'  
python --version  
python -m pip install -e .
```

If python is not recognized, try py -3 --version. Use py -3 in place of python throughout if that is your working interpreter, or use an installed Python executable's absolute path with PowerShell's & operator. Install into the same interpreter you will use to launch the app.

The editable installation allows python -m alpaca_agents... to find the package. Runtime code uses the Python standard library. Node is needed for the browser-side test suite, not normal trading operation.

Keep credentials out of documents and chat

The executor reads ALPACA_PAPER_API_KEY and ALPACA_PAPER_API_SECRET. Market data reads MASSIVE_API_KEY or POLYGON_API_KEY. Optional generative chat reads ANTHROPIC_API_KEY. A .env file is not automatically loaded by this application.

If you already maintain a protected local key-loading script, load it into the terminal you will use. The path below is the project's documented convention; it is not proof the file exists or is current.

```
. 'C:\Users\eveli\keys.ps1'
```

Do not print the environment, paste keys into agent chat, include them in screenshots, or save them in this repository. A process receives its environment at launch; after updating credentials, restart the affected process. Prefer separate shells with only the credentials needed for research, broker diagnostics, or combined controller operation.

Initialize controls only for a new setup

The following creates the directory and writes DISABLED. Do not blindly overwrite an existing operator-selected mode, especially if positions are held: DISABLED also prevents automatic exits.

```
python -c "from pathlib import Path; Path('runtime').mkdir(exist_ok=True)"  
@'  
from pathlib import Path  
Path('runtime/trading-control').write_text('DISABLED', encoding='utf-8')  
'@ | python
```

Python writes predictable UTF-8 marker files. Avoid relying on PowerShell redirection, whose encoding differs by version. The exact trimmed marker text matters.

Margin-paper acknowledgement

If reconciliation identifies a margin paper account, the current implementation supports an explicit acknowledgement marker. It still enforces its local cash semantics and \$2,000 capital cap. This does not convert the brokerage account into a cash account or waive broker rules. Only create the marker after understanding that distinction.

```
@'  
from pathlib import Path  
Path('runtime/paper-margin-acknowledged').write_text('ACKNOWLEDGED', encoding='utf-8')  
'@ | python
```

4. Open the dashboard in the right mode

Static mode: saved evidence, no server

Double-click Open-Dashboard.cmd, or generate and open the dashboard manually:

```
python -m alpaca_agents.dashboard  
Start-Process .\runtime\dashboard.html
```

The launcher rebuilds the HTML from local records. It does not reconcile with Alpaca, scan markets, or submit orders. Static chat is a keyword-based guide over the embedded snapshot. Refreshing an old HTML file does not create a new broker check; rebuild the file after records change.

Served mode: fresh local reads per question

```
python -m alpaca_agents.controller --runtime runtime --serve
```

Open the localhost URL printed in the terminal and keep that terminal open. The port is selected at launch. This mode reads current local records when you ask a question; those records may still describe old broker observations. No periodic cycles run unless --every is supplied.

Select Houston and send the exact message reconcile to request a dry controller diagnostic. That request can contact the broker, import activity, resolve eligible terminal records, and write diagnostics, but it cannot submit orders or reserve new entries through the chat path. It needs the appropriate process credentials. Its ability to scan also depends on control mode and market-data access.

Scheduled served mode

```
python -m alpaca_agents.controller --runtime runtime --serve --every 300
```

This adds scheduled cycles without order submission. It owns the same runtime lock as a terminal-only controller. Do not launch another controller on that runtime. A served schedule can pause at market close or on an incident while the browser server remains available. Restart the session deliberately after checking the cause; do not assume it resumes tomorrow.

Read the dashboard in this order

1. Overview: control mode and recorded state. Check data-quality warnings before trusting numbers.
2. Activity: latest cycle timestamp and reconciliation reasons. Recent page activity is not a recent broker check.
3. Positions: remaining ledger inventory and basis. Basis is not a current mark-to-market portfolio value.
4. Outstanding intents and risk checks: unresolved work, loss latch, and entry blockers.
5. Scanner: source freshness, errors, shadow ideas, and skips.
6. Research: report dates, samples, mean and median R, and downside observations.
7. Chat: ask for an explanation of specific records you have already identified.

The served browser polls metadata about every 30 seconds. That updates connection/age indicators, not every main panel. Refresh the page to rebuild the displayed panels from current local records. Unknown means unavailable evidence; it is not the same as an empty account or zero loss.

5. Research before enabling anything

Ask a testable question

A useful question is: "Does this unchanged trend definition show reasonably consistent underlying-price outcomes across the chosen period and symbols?" A weak question is: "Which combination produces the biggest number?" Write down the universe, dates, playbook, and acceptance criteria before comparing results.

The backtest names are trend, oversold_bounce, and breakout. Controller playbook names are different: trend_directional, trend_debit_spread, oversold_bounce, and breakout_continuation. Research on trend does not establish that an executable option contract will satisfy the premium and liquidity limits.

Fetch once and save a reproducible cache

Both --start and --end are required for a network history run. These dates are an example research window ending at the previous completed session for this guide's date; choose your own completed period.

```
python -m alpaca_agents.backtest --symbol QQQ `
--start 2023-01-01 --end 2026-09-17 `
--playbooks trend oversold_bounce breakout `
--save-bars runtime/bars-QQQ.json `
--output runtime/bt-QQQ.json
```

PowerShell's backtick must be the final character on its line. A single-line command is equivalent. Repeat for other supported ETFs when the research question calls for them.

Repeat from the same data

```
python -m alpaca_agents.backtest --symbol QQQ `
--bars-file runtime/bars-QQQ.json `
--playbooks trend oversold_bounce breakout `
--output runtime/bt-QQQ.json
```

This run needs no market-data request. Keep the source bars and report together. Use bt-*.json output filenames for dashboard discovery; the CLI's default backtest-QQQ.json does not match that pattern. Generative chat's symbol lookup specifically uses bt-QQQ.json, so retain a canonical report per symbol. Archive experiment variants separately to avoid accidental duplicate comparison.

Read the full result, not only expectancy

Begin with resolved count and exclusions. Then read mean R, median R, profit factor, worst loss R, win rate, holding time, and exit reasons together. Fewer than 30 resolved observations fails the code's minimum-sample flag. Thirty is a minimum software threshold, not statistical proof.

An R-multiple measures the underlying move relative to the modeled stop distance. For an illustrative long entry at 100, stop at 98, and exit at 103, the underlying return is +1.5R. That does not mean an option made \$150, 150%, or any predictable premium return. Option prices, IV, theta, bid-ask costs, and fees are not modeled.

Read the caveats and trades array as well as summary. A profitable stop-rule exit is possible: outcome is the sign of R; exit reason describes which condition ended the trade. no_fill, no_trade, and unresolved are different categories, not ordinary losses.

6. Run a genuine shadow-testing phase

Separate connectivity testing from signal testing

With trading-control set to DISABLED, the controller reconciles but skips the entry/scanner stage. Older operator notes describe a disabled loop as if it scanned everything. To study ideas while keeping the control file disabled, use the standalone shadow scanner separately.

First check broker connectivity and reconciliation:

```
python -m alpaca_agents.executor account
python -m alpaca_agents.executor reconcile
```

account reports selected broker fields. It does not verify settled cash, history coverage, or inventory consistency. reconcile returns detailed reasons and a snapshot valid for the rules engine only within 60 seconds. A successful check authorizes no order by itself.

Outside a session, MARKET_CLOSED or NOT_A_SESSION can be expected. Do not dismiss additional reasons simply because the market is closed. During an open session, success means reconciled: true and an empty reasons list.

Run the standalone scanner

```
python -m alpaca_agents.scanner --session 2026-09-17 `
--symbols SPY QQQ IWM --output runtime/shadow-scan.json
```

Replace the example date with the last completed exchange session. The scanner combines completed daily bars with current eligible option quotes; setting an old --session does not create an options historical backtest. Use it during an active market session when fresh quotes are available.

It cannot enable playbooks or place orders. Its report contains generated_at, sources, errors, proposals, shadow, and skipped. Snapshot errors make the command return a failure status even if other symbols were processed. Inspect individual symbols; do not reduce the result to one aggregate count.

Interpret the outcome correctly

- An error means the scanner could not establish valid inputs. No strategy conclusion follows.
- A valid skip means the data was processed but a signal, structure, price, liquidity, or portfolio condition failed.
- A shadow idea is a candidate that survived its scanner conditions while disabled for execution. It has not been approved for a trade.
- An enabled proposal is still subject to deterministic risk checks, journal validation, and broker submission gates.

Stocks data access does not imply access to current option quotes. Verify that your current Massive subscription supplies the exact snapshots, quotes, and freshness the code requires. Do not buy a plan solely because an old project note names a tier or price. Provider documentation is linked in Volume 3.

Keep a shadow log

For each session record: date, run time, bars session, symbols attempted, successful snapshots, errors, skips, ideas, and whether you understand each decision. Save interesting reports under dated filenames before the default report is overwritten. Rebuild the dashboard after a new scan.

Ten clean sessions is a useful project practice target, not an automatic promotion rule. Include sessions with no signals. If options data is unavailable, continue underlying research and local dashboard learning, but do not count those days as successful option-scanner validation.

7. Know exactly what can become an order

Signal and contract filters

The trend signal uses EMA20/EMA50 ordering and 20-session momentum, with pullback or prior-bar-break logic. It rejects chasing more than its configured distance and stops narrower than 0.5 times ATR14. The bounce signal looks for an oversold condition within a long-term uptrend. Breakout uses a tight range near highs and a volume-confirmed break.

Contract selection currently requires 30-45 calendar days to expiry, positive two-sided prices, open interest of at least 500, and a bid-ask width no greater than 10% of the midpoint. Long options target absolute delta near 0.40 within 0.35-0.45. Eligible long entries are costed at the ask. Quotes must pass the adapter's freshness and metadata checks.

These filters can make qualifying contracts rare. A strong underlying signal does not guarantee an affordable 30-45 DTE option. Do not widen limits merely to make the activity panel less empty.

Risk is total debit plus estimated fees

The entry rule is $\text{limit_debit} \times 100 \times \text{quantity} + \text{estimated_fees} \leq \100 . Exactly one contract, or one validated spread unit at the rules layer, is allowed; the executor currently rejects spreads. The scanner's \$0.65 per-contract fee estimate is a placeholder, not a current broker tariff or a verified round-trip cost.

Illustration: a \$0.90 quoted option with one contract and \$0.65 estimated fees uses \$90.65 of modeled entry risk. A \$1.00 premium plus those fees is \$100.65 and exceeds the cap. Do not confuse the option's per-share quote with its 100-share contract cost.

Shared account constraints

The rules allow at most two open plus pending entries, reject another entry after five attempts in a rolling hour, and require sufficient reconciled settled cash. The controller considers at most one new entry per cycle and prevents a new idea for an underlying already held or pending. Spendable cash is constrained by the local \$2,000 capital cap minus open basis as well as settlement checks.

The daily breaker uses cumulative realized losses and relevant fees, without letting wins erase losses. For illustration, $-\$25, +\30 , then $-\$20$ gives $+\$5$ net but $\$45$ gross losses before any additional fees. The loss gate has been crossed. It blocks new entries; it does not cap total losses on open positions at \$40.

Readiness requires all gates together

New paper orders require an approved playbook marker, the matching `--enable-playbook` launch flag, `ARMED_PAPER`, `--submit`, fresh reconciliation, eligible data/structure, and a successful journal authorization. A green dashboard component proves only that component. A successful backtest is not an approval marker; an approval marker is not an active launch flag.

8. Conduct a controlled paper pilot

Prepare a written decision first

Choose one executable playbook and state the symbol universe explicitly. Explain what evidence supports the test, what remains unknown, what would stop the pilot, and when it will be reviewed. The example below uses `trend_directional` and `QQQ` solely to demonstrate syntax; it is not a recommendation to trade `QQQ` or an assertion of an edge.

Stop an existing controller cleanly before changing launch configuration. Its approved/enabled playbook set is loaded at startup; do not rely on editing approval files mid-run to reconfigure it. The trading-control file is checked during operation and is the immediate control mechanism.

Approve and select the mode deliberately

These commands change local trading permissions. Use them only after completing your own review, with the controller stopped and the intended runtime directory confirmed.

```
@'
from pathlib import Path
Path('runtime/playbooks').mkdir(parents=True, exist_ok=True)
Path('runtime/playbooks/trend_directional.approved').write_text('APPROVED', encoding='utf-8')
'@ | python
@'
from pathlib import Path
Path('runtime/trading-control').write_text('ARMED_PAPER', encoding='utf-8')
'@ | python
```

Optional dry entry-cycle check

```
python -m alpaca_agents.controller --runtime runtime `
--symbols QQQ --enable-playbook trend_directional --dashboard
```

No `--submit` means no POST, but this is not purely read-only: an approved candidate can create a temporary entry reservation that expires. The current authorization TTL is 30 seconds. If there is no candidate, an empty entry list is expected. A dry run does not prove a future order will fill.

Launch the supervised paper session

```
python -m alpaca_agents.controller --runtime runtime `
--symbols QQQ --every 300 --dashboard `
--enable-playbook trend_directional --submit
```

This command can submit paper orders. Keep the terminal open, confirm initial reconciliation, and inspect every first-session order transition. To add browser chat to the same process, include `--serve`. Do not run a separate served controller on the same runtime.

The five-minute setting is a polling interval, not an exact wall-clock schedule. Work takes time, the loop can stop, and the computer can sleep. There is no built-in daily auto-start service. Verify the next day's launch instead of assuming yesterday's terminal still manages positions.

Verify outcomes rather than inferring them

A submitted response is not a fill. Check broker order status, imported fills, local inventory, and the next reconciled cycle. Do not click repeatedly or rerun a manual order command after an ambiguous response. Preserve the `client_order_id` and request trace for investigation.

9. Understand how exits really work

Priority after the entry session

Automatic exit evaluation uses this order: underlying stop, underlying rule such as an EMA20 close-through, premium stop, underlying target, then time stop. The first matching rule supplies the recorded reason. Multiple conditions can be true at once; a different recorded reason is not necessarily an error.

Underlying conditions use the last completed session's close, not a continuously observed intraday price touch. The premium check uses the broker option mark at evaluation time. This is a polling exit manager, not a resting broker stop order.

Entry-day behavior is narrower

On the session a position was opened, only the premium stop is evaluated. There is no same-session target or time exit from these automatic rules. The code permits this protective same-session exit only while fewer than three counted day trades have been used in its rolling five-session window.

Treat that as this version's implemented policy. It is not a complete statement of current broker eligibility or trading regulation. If the budget is exhausted, the code can hold the position despite a premium-stop condition; it does not guarantee a 50% maximum loss.

Automatic pricing versus manual flatten pricing

Automatic exits use a fresh, positive, two-sided option bid when available. Otherwise they can use a day-limit price at 95% of the broker mark, floored at \$0.01, provided the necessary inputs survived collection. A provider exception can instead leave exit inputs unavailable. Read the cycle record to distinguish a fallback price from a failed input stage.

The manual flatten command uses 95% of the broker mark; it does not retrieve the automatic manager's fresh bid. Both are limit orders. Neither pricing method guarantees execution. The application has no general cancel/replace or chase mechanism.

Time stops differ from backtest assumptions

The scanner's long-option plans use a 21-DTE time stop. The oversold_bounce plan also includes a ten-session limit. The trend_directional plan does not currently include the backtest's fifteen-session holding limit. This is an important implementation difference: replay holding-time assumptions are not automatically copied into execution plans.

An unfilled day order is not a closed position. An exit can be reevaluated in a later session with a fresh decision key, while outstanding or uncertain intents must still reconcile. Never assume a position is flat because an exit notification appeared.

Keep exit management operating when winding down

EXITS_ONLY prevents new entries while allowing exit handling. It still needs a running submission-enabled controller, valid inputs, and acceptable state. DISABLED prevents automatic exits as well as entries. Neither setting cancels an order already at the broker or reverses a fill.

10. Build an efficient daily and weekly routine

Before starting a session

Confirm the correct account/runtime pair, machine availability, current credentials, intended control mode, and launch flags. Review open positions and unresolved intents before looking at new ideas. Check that yesterday's controller stopped cleanly. If the market is open, run or inspect a fresh reconciliation and resolve every unexpected reason.

Pick the activity appropriate to your phase: cached research, standalone shadow scanning, disabled connectivity loop, or a reviewed paper pilot. Keep one concise session note that records what you intended to test. That makes unexpected activity easier to identify.

During a supervised pilot

Check the first cycle, then each initial entry/exit lifecycle. Watch timestamps and notifications, especially after a network interruption, laptop sleep, credential change, or subscription change. Once behavior is established, use scheduled check-ins appropriate to the system's polling frequency and the positions held; do not assume the dashboard is an alerting service.

If you need to step away, decide how positions will be managed before closing the terminal. Closing a browser tab does not close the controller. Closing the owning terminal generally stops the process and its automatic exit checks.

End-of-session review

```
Get-Content runtime\cycles.jsonl -Tail 3 |  
  ForEach-Object { $_ | ConvertFrom-Json } |  
    ConvertTo-Json -Depth 12  
Get-Content runtime\notifications.jsonl -Tail 15
```

Review reconciliation, exits, scan counts when present, entry decisions, and submission results. The actual entry records are under stages.entries.entries; scanner counts are under stages.scan. Not every stage exists in a halted or control-skipped cycle.

For each closed trade, document the original idea, accepted risk, actual fills, exit reason, costs, delays, and deviations. For each no-trade day, document whether it was a clean absence of candidates or an operational problem. Preserve rejected and failed cases; they are part of the denominator.

Weekly review

Separate process reliability from strategy observations. Count attempted sessions, clean sessions, data failures, unexplained reconciliation failures, order ambiguities, unfilled orders, and closed trades. Evaluate option results from actual fills and costs; do not substitute underlying backtest R.

Keep the playbook and limits unchanged during the observation window. If you make a change, record its date and begin a distinct cohort. A small-sample good week should not justify higher size, and a bad week should not trigger unrecorded parameter changes. Volume 2 includes reusable review templates.

11. Limits, evidence, and the next learning cycle

Known implementation gaps to remember

Multi-leg execution is unavailable even though the scanner and rules understand some spreads. Assignment and exercise activities are not a fully supported lifecycle. Worthless expiration has a normalization path, but the exact broker records still need to match supported shapes. Unknown activity or mismatched inventory blocks reconciliation.

The underlying replay excludes fills beyond stop/target, while the repository documents no equivalent current intraday underlying-entry recheck in execution. A daily signal can therefore be stale relative to the next session's movement. Treat this as a material research-to-execution gap when reviewing candidates and deciding whether the implementation is ready for a pilot.

Backtests use daily bars and simplified fills. They do not model option premiums, IV, theta, fees, or options liquidity. Automated target checks use completed closes, whereas replay can resolve targets from intraday daily-bar highs/lows. Time-stop assumptions also differ as described in chapter 9. Do not interpret replay performance as execution parity.

A useful decision record

Write: the question; code revision; bars and report filenames; test dates; playbook and universe; key statistics; operational defects; unresolved assumptions; decision; and next review date. Distinguish "reject this test configuration," "collect more evidence," and "eligible for a bounded paper pilot." Those are different conclusions.

The historical QQQ/SPY/IWM figures in the handover are useful audit context, not a current ranking or recommendation. Reproduce results from the intended data and current code. If a result changes, compare the inputs, source revision, and exclusions before explaining it as market behavior.

Your best next action

If setup is new, finish chapters 3-4. If you can already open the dashboard, reproduce one cached report and complete the shadow log in chapter 6. If you already have paper positions, prioritize fresh reconciliation, the exit-management state, and the incident reference over new strategy experiments.

Primary local sources: `controller.py`; `rules.py`; `scanner/scan.py`, `signals.py`, `contracts.py` and `__main__.py`; `executor/reconcile.py`, `orders.py`, `exits.py` and `__main__.py`; `dashboard.py`; `studio.py`; `llm_chat.py`; `backtest/__main__.py` and `replay.py`. All module paths are under `src/alpaca_agents/`. The companion reference includes external provider links and a source map.

AlpacaAgent in practice

Use cases, agent prompts, and review templates

Worked workflows for learning the system, interpreting research, investigating decisions, and running a disciplined paper experiment.

DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

1. Get better answers from the agents	3
2. Use case: understand the workspace in ten minutes	4
3. Use case: get a market briefing without mixing it with signals	5
4. Use case: compare research without fooling yourself	6
5. Use case: investigate why there are no trades	7
6. Use case: inspect a candidate from idea to refusal	8
7. Use case: understand a held position and its exit	9
8. Use case: pause, leave, or recover after an interruption	10
9. Use case: manage generative chat efficiently	11
10. Templates for disciplined operation	12
11. Weekly review and progression worksheet	13

1. Get better answers from the agents

This volume accompanies the operator's handbook. It describes source revision 235b087, reviewed September 18, 2026. Prompts below are questions for the application; they do not create new capabilities. Examples containing numbers are synthetic unless explicitly labeled otherwise.

First identify which chat you are using

Static dashboard chat is a deterministic keyword router. It chooses a canned explanation based on the embedded snapshot. It does not reason through a long prompt, remember a research plan, or execute a command. Use short topic questions and the suggested chips.

Served non-generative chat uses the same type of router but rereads current local records for each request. The exception is Houston's exact reconcile message, which requests a dry diagnostic cycle. "Please reconcile everything and then trade" is not the exact diagnostic command and cannot trade.

Generative chat is optional in served mode when ANTHROPIC_API_KEY is available at launch. It can use four local tools and two market-data tools: `read_snapshot`, `read_backtest`, `read_cycles`, `explain_rule`, `read_market`, and `read_news`. Market tools require the server's Massive/Polygon credentials and suitable access. They retrieve stock snapshots and headlines, not live option chains.

A useful prompt has four parts

Name the question, identify the evidence, specify the output, and require uncertainty to remain visible. For example:

Astra: Read the latest local snapshot and recent cycles. List the three most important blockers, the source and timestamp for each, and the next diagnostic step. Keep missing records separate from confirmed zero values.

Keep prompts focused. The current generative implementation sends at most 800 characters of user text per question and retains a bounded per-agent history of 20 user/assistant messages. Long pasted reports are a poor interface; ask the tools to read their supported records. Restate key context when switching agents.

What to look for in an answer

A good answer identifies the source, date, missing values, and whether it is describing local records or newly fetched market information. A fluent answer without record references is not a verified state check. If a response falls back to rule-based mode, the source text should explain that. Reformulating the same question will not repair an unavailable provider or missing data entitlement.

Do not paste credentials into chat. In generative mode, your message, bounded conversation history, and sanitized tool outputs can be sent to the configured Anthropic API. Local hosting does not mean every answer is computed entirely offline.

2. Use case: understand the workspace in ten minutes

Goal and prerequisites

You want to understand what the system last did without starting paper trading. You need the project installed and whatever local runtime records already exist. Broker and market-data credentials are not necessary merely to open the workspace.

Procedure

1. Run `python -m alpaca_agents.dashboard` and open `runtime/dashboard.html`, or start `controller --serve` and open its printed URL.
2. Check the rendered timestamp and latest cycle timestamp. They answer different questions: when the page was created, and when the controller last recorded work.
3. Look for missing/unreadable record warnings. An unknown ledger prevents conclusions about an empty account.
4. Read the control mode, open positions, and outstanding intents. DISABLED with a held position deserves immediate attention because automatic exits are disabled too.
5. Read recent reconciliation reasons and notifications before interpreting research or scanner cards.
6. Write a three-line summary: confirmed facts, unknown facts, and the next useful diagnostic.

Prompts

Rule-based: "Give me a briefing." Then ask separately: "What are the blockers?", "What positions are recorded?", and "What do the controls mean?"

Generative:

Astra: Summarize the saved workspace state. Show control mode, latest cycle time, reconciliation outcome, held positions, outstanding intents, and missing records. Do not infer a running controller from a recent timestamp.

Houston: Explain the outstanding intents in the local snapshot. Distinguish reservations from claimed orders and say what evidence is missing before any operational decision.

Example interpretation

Suppose the page was rendered at 14:00 UTC, the last cycle finished at 12:00 UTC, and the mode is DISABLED. You know the page was built recently from two-hour-old cycle evidence. You do not know that the broker was checked at 14:00 or that a controller is currently running. If you need broker state now, a fresh diagnostic is the next useful action.

Completion criterion

You can name the files or timestamps supporting each statement. You have not equated a polished overview, a successful browser connection, or an agent's confident tone with fresh trading readiness.

3. Use case: get a market briefing without mixing it with signals

Goal and prerequisites

You want context on the current market and a separate explanation of what AlpacaAgent did. Use served generative mode with an Anthropic key and suitable Massive data access. Static or deterministic chat cannot fetch a fresh market briefing.

Recommended prompt sequence

Star: Use read_market for SPY, QQQ, IWM, and DIA and read_news for recent headlines. Give each snapshot's timestamp, the reported moves, and attributed headlines. Separate these observations from the system's saved scanner results. Do not turn a headline into a trade signal.

Astra: Now read recent cycles and explain what the system actually did today. Separate successful checks, missing data, skipped candidates, and submitted-order records. Identify the latest timestamp for each claim.

How to evaluate the response

Stock snapshots establish stock-market observations only. They do not show that the scanner found an eligible 30-45 DTE contract under the risk cap. Headlines are third-party reports; a short provider description is not an independently verified explanation for a price move.

If one symbol has no available snapshot, the right result is a missing-data statement. If market data is unavailable entirely, ask for an explanation of saved local records or a general concept instead. Do not prompt the model to guess today's price from memory.

Follow-up questions that help

Nova: Explain the difference between an underlying stock move, an option premium move, and an underlying backtest R-multiple. Use hypothetical numbers and label them as hypothetical.

Star: Which parts of this briefing are retrieved data, which are attributed news, and which are interpretation? What option-specific information would still be needed to evaluate a candidate?

What this workflow cannot do

The chat tool does not fetch a live option chain or create a trade authorization. A report that "QQQ rose" does not mean trend_directional fired, the contract was liquid, Moon approved it, or Houston sent an order. Follow the evidence from each layer instead of collapsing the story into one bullish or bearish conclusion.

Completion criterion

You have a timestamped briefing and a separate operational summary. You can explain what is known about the market without treating it as an instruction to enable or submit a trade.

4. Use case: compare research without fooling yourself

Goal

Compare the same playbook across symbols and determine whether the evidence is worth further testing. Keep data dates, code revision, and modeling assumptions consistent. Use the cached-bars workflow from Volume 1 to make comparisons reproducible.

A synthetic comparison

Measure	Sample A	Sample B
Resolved observations	40	38
Mean R	+0.25	+0.90
Median R	+0.15	-0.35
Profit factor	1.45	2.20
Worst loss R	-1.80	-7.50

Sample B has the larger mean but a negative typical observation and a much worse downside result. It deserves an outlier and concentration review, not automatic selection. Sample A is more internally consistent on these few measures, but neither table models option returns or proves a persistent edge.

Procedure

1. Check `first_bar`, `last_bar`, `bars`, `source`, and `generated_at`. Exclude accidental comparisons of different periods.
2. Read `resolved`, `no_fill`, `no_trade`, and `unresolved` categories. A small number of resolved trades after many signals is informative.
3. Compare mean and median together. Inspect the largest winners and losers in the trades array.
4. Read `win_rate` separately from exits. A stop exit can be profitable; a timeout can be a win or loss.
5. Split your review by period or market regime where you can do so without repeatedly choosing the most flattering window.
6. Decide whether to reject, retain as inconclusive, or study in shadow mode. Record why before changing parameters.

Generative prompts

Star: Read the QQQ backtest. Report the data window, resolved count, mean R, median R, profit factor, worst loss R, `no_trade` count, and model caveats. Explain what conclusions remain unsupported.

Moon: Explain why a positive underlying expectancy does not establish option profitability. Relate the missing option-price model to the system's actual risk and contract filters.

The chat tool exposes a report summary; inspect the JSON locally for trade-level calculations. Finish with a decision that includes favorable and unfavorable evidence, sample limitations, and the exact report. Do not convert R to dollars or select solely on mean R.

5. Use case: investigate why there are no trades

Goal

Identify the first missing condition without changing it simply to produce activity. "No trades" can be the correct result at many layers.

Follow this diagnostic sequence

1. Is the controller actually running? Check its terminal and latest cycle time. Opening the dashboard does not start it.
2. Did reconciliation pass? If not, read every reason and stop investigating hypothetical entries until state is understood.
3. What is the control mode? DISABLED and EXITS_ONLY skip the controller's entry stage. Use the standalone scanner for disabled shadow research.
4. Did the scanner obtain valid snapshots? HTTP errors, missing Greeks, stale quotes, and malformed data are not evidence of absent setups.
5. Did a signal pass? No trend, excessive chase, tight stop, or weak reward:risk can correctly eliminate it.
6. Was a contract eligible and affordable? DTE, delta, open interest, bid/ask width, and total premium can remove an otherwise valid signal.
7. Was the matching playbook approved and enabled at launch? A marker by itself is insufficient.
8. Did Moon or the journal reject the candidate? Look for the exact reason, not a paraphrase such as "risk issue."
9. Was --submit present? Without it, an ordinary enabled dry cycle can reserve but will not submit.
10. Was an order submitted but unfilled? That is an execution result, not an absence of proposals.

Prompts

Astra: Using the latest cycles and snapshot, identify the earliest confirmed blocker in the path from reconciliation to submission. Say which later stages have no evidence because the cycle stopped early.

Star: Explain the recorded scan errors and skips. Distinguish invalid inputs from valid no-signal outcomes. Do not suggest loosening a filter simply to get a candidate.

Moon: Explain the exact rejection reason in the latest entry record and the invariant it protects. If the record is unavailable, say which source is missing.

Worked example

Assume reconciliation passed and control is DISABLED. The correct controller record can say entries were skipped for that mode, with no scan stage. That is expected. If a separately run shadow scan returns three HTTP 403 errors, the next task is verifying data access. Approving a playbook or adding --submit would not repair those snapshots.

Completion criterion

You have one evidence-backed explanation for the earliest blocker. Your next step tests that explanation rather than changing several independent settings at once.

6. Use case: inspect a candidate from idea to refusal

Synthetic candidate

Suppose the underlying entry level is 100, stop 98, and target 103. Direction is long. A one-contract call has a quoted ask of \$0.90 and estimated fees of \$0.65. The underlying reward:risk is $3 / 2 = 1.5$. Modeled entry risk is \$90.65.

This arithmetic is necessary but insufficient. The option must also have trusted metadata, eligible expiry and delta, sufficient open interest, acceptable bid/ask width, and current quotes. The account must have fresh reconciled state, capacity, cash, and no entry breaker. The journal must support the structure and confirm the playbook gates.

Walk through each layer

Star should provide a thesis tied to the actual signal, the option structure, levels, premium, and an exit plan. A score is a ranking input, not a calibrated probability of success.

Moon recomputes the level relationship and risk. If the ask were \$1.00 with the same fee estimate, the idea would exceed \$100 even though the chart setup was unchanged. If two positions or pending entries already occupy capacity, an affordable idea can still be refused.

Houston uses current broker and journal evidence. The fact that the rules layer can validate a debit spread does not make that spread executable. The journal currently refuses unsupported execution structures.

Prompts

Star: Explain the recorded candidate's thesis, entry, stop, target, chosen structure, and quote timestamp. Separate observed fields from any inference. Do not invent missing quotes.

Moon: Using the recorded fields, explain the risk formula and each applicable rejection condition. Do not treat the scanner's score as a success probability.

If the relevant detailed idea is not exposed to chat, open the shadow report or cycle file directly. Agent prompts cannot expand the read tool's schema.

A useful paper journal entry

Record the candidate's symbol and playbook, signal session, observation time, underlying levels, OCC contract, bid/ask, expiry, delta, total estimated risk, and final disposition. A rejection should retain its reason. A submission should retain its client_order_id and actual fill outcome.

Completion criterion

You can explain why the idea advanced or stopped at each layer. You have not treated one layer's approval as an executable order or assumed that a theoretical stop determines the option's realized loss.

7. Use case: understand a held position and its exit

Goal

Understand what the exit manager actually observed and why it held or exited. Use the journaled entry idea, fill inventory, and cycle's exits records. The dashboard's basis is not a current option quote.

Inspect in this order

1. Identify the exact contract and entry session. Confirm broker and ledger inventory reconcile.
2. Check control mode and whether a submission-enabled controller is operating.
3. Read the entry idea's `exit_plan`. Do not substitute a playbook summary from memory.
4. Check the option mark, available bid, underlying completed close, and any bars needed for EMA20.
5. Read the exit decision or hold note in the latest cycle. Missing inputs require a different response from a valid hold decision.
6. If there is an order, verify its status and fill evidence. Keep monitoring until the position and intent resolve.

Synthetic premium-stop example

For a single contract with \$90.65 remaining basis, a 50% premium-stop threshold is approximately \$45.33 after cent rounding. A broker mark of \$0.40 represents \$40 per standard contract and crosses that threshold. The actual exit proceeds depend on whether a priced limit order is submitted and filled.

On the entry session, the code checks only this protective premium condition and its day-trade budget. On later sessions, an underlying stop or EMA20 rule may have priority. A premium-stop notification therefore does not establish a fixed final loss percentage.

Prompts

Houston: Explain the latest recorded exit decision for the held contract. State the inputs, their source, the selected rule, whether an order was prepared or submitted, and what remains unknown about its fill.

Moon: Explain the priority of automatic exit rules and the entry-day exception. Keep the implemented day-trade policy separate from any claim about current broker rules.

When the answer is "no exit inputs"

Do not assume the fallback mark was used. The controller can discard the input set after a provider exception, leaving no usable price. Inspect notifications, confirm the broker position, and follow the manual-management reference if intervention is necessary. A scheduled loop does not guarantee that each position received a valid exit evaluation.

Completion criterion

You can distinguish a hold, a condition that fired without a price, a prepared dry decision, a submitted limit, and an actual fill. Only the last changes position exposure at the broker.

8. Use case: pause, leave, or recover after an interruption

Goal

Make a deliberate operational transition without assuming that changing a local mode cancels broker activity. Begin with open positions and outstanding orders, not the desired appearance of the dashboard.

If you only want to stop new entries

EXITS_ONLY is the appropriate local control for continued exit management. It requires the controller to remain running with --submit and usable inputs. Confirm a subsequent cycle reads the intended mode and handles held positions. A closed browser tab is harmless to the server process; a closed owning terminal is not.

If you want to stop the entire application

Confirm how every position and pending order will be managed first. DISABLED stops automatic entries and exits but does not undo fills or cancel broker orders. A manual flatten attempt is a new exit-order workflow and needs its own outcome verification.

After sleep, power loss, or a process crash

1. Check the broker for current positions and order status before interpreting stale local records.
2. Confirm the old controller process is gone. A leftover controller.lock is not proof that the process is alive, but deleting it while the process runs can permit concurrent ownership.
3. Preserve the runtime evidence. Do not reset databases to make the app start cleanly.
4. If the lock is truly stale, remove only that lock as described in Volume 3.
5. Restart in a controlled diagnostic mode, reconcile, and inspect unresolved intents.
6. Resume the intended schedule only when state is understood. Do not retry an ambiguous order because the terminal disappeared.

Prompt

Houston: Read the saved order and cycle records. Identify unresolved intents and the last confirmed outcomes. State what must be checked at the broker before resuming. Do not infer that a missing local completion means the order failed.

Completion criterion

There is a known owner for position management, broker and local state are reconciled or the incident remains explicitly open, and no duplicate submission has been used as a recovery shortcut.

9. Use case: manage generative chat efficiently

Choose the right agent and question size

Use Astra for a broad operational summary, Houston for order and reconciliation records, Moon for deterministic constraints, Star for scanner/research interpretation, and Nova for general concepts. The personalities help organize questions; they do not create different permissions.

Ask one bounded question at a time. A useful sequence is "show facts," "explain one discrepancy," then "summarize the next diagnostic." This tends to produce more checkable results than asking for a complete market outlook, trade recommendation, and account audit in one message.

Understand the cost controls

ALPACA_AGENTS_LLM_DAILY_CAP defaults to 100 and is clamped between 0 and 2000. It counts provider calls, not messages or dollars. One answer can require multiple calls through tool rounds. The check occurs before the question's call sequence, so the setting is not a strict per-request dollar ceiling and a tool-using answer can cross the count threshold.

Usage and chat history are held in memory. Restarting the process resets that in-memory accounting. Use the provider's account controls for durable spending limits. The source default model string is claude-sonnet-4-5; current availability is a provider question, not something this guide verifies. ALPACA_AGENTS_LLM_MODEL is the code's override variable.

Save useful conclusions outside chat

Conversation history is bounded and cleared through the UI or process lifecycle. It is not the trade journal. Copy a concise factual conclusion into your operator notes with the source record and timestamp, without keys or private provider tokens. Do not assume another agent remembers the first agent's context.

If an answer is repetitive or missing

Check whether you are in rule-based mode, whether the source reports a provider failure or budget fallback, and whether the requested tool actually exposes the record. A generic reply may be the deterministic fallback, not evidence the model reviewed the data. Return to the underlying JSON or terminal diagnostic when necessary.

Compact prompt library

- Astra: "What changed in the last three recorded cycles? Cite times; keep unknowns explicit."
- Houston: "Which intents are still unresolved, and what is the last confirmed broker evidence?"
- Moon: "Explain why positive net P&L does not clear the cumulative-loss breaker."
- Star: "Read the QQQ summary and explain mean/median disagreement and sample limitations."
- Nova: "Explain option bid, ask, midpoint, mark, and cost basis using hypothetical numbers."

10. Templates for disciplined operation

Session plan

Date and intended time window: _____

Phase: research / shadow / diagnostic / reviewed paper pilot

Code revision and runtime directory: _____

Question being tested: _____

Playbook and symbol universe: _____

Expected control mode and launch flags: _____

Known positions and unresolved intents: _____

Stop conditions and next review time: _____

Shadow observation

Run time and completed bars session: _____

Symbols attempted / valid snapshots / errors: _____

Signal skips versus contract skips: _____

Candidate thesis, levels, contract and quote time: _____

Could I explain the result before reading the agent's prose? ____

Evidence filename and follow-up: _____

Trade lifecycle review

Playbook, underlying, OCC contract and signal session: _____

Entry authorization/client order ID and decision reason: _____

Estimated entry risk / actual debit / actual fees: _____

Order submitted time / status / fill time and quantity: _____

Exit rule / inputs / chosen limit / fill outcome: _____

Net realized result and gross loss-counter effect: _____

Operational deviation, if any: _____

Lesson and whether it requires a separate experiment: _____

11. Weekly review and progression worksheet

Review operational reliability first

Count planned sessions, completed sessions, successful reconciliations, scanner input failures, unexplained position/order mismatches, unknown submission outcomes, and exits lacking required inputs. Record whether each incident has a root cause and a verified resolution. A good profit result does not erase an unresolved execution defect.

Review strategy observations second

Count clean candidates, refusals by category, actual entries, unfilled entries, actual closes, and unresolved positions. Summarize actual option P&L and known costs separately from historical underlying R. Keep separate cohorts when playbook logic, symbols, data plan, fee assumptions, or operating procedure changed.

Write a decision in five sentences

1. This week's intended experiment was: _____
2. The reliable evidence we collected was: _____
3. The most important uncertainty or failure was: _____
4. We will continue / pause / reject / revise because: _____
5. The next review will occur after: _____

Prefer evidence-based milestones over a fixed calendar promise. A month with two trades is not equivalent to a month with thirty closed observations, and neither is enough if the records are inconsistent. A sparse strategy may require a long observation period; forcing activity changes the experiment.

Suggested learning sequence

First, demonstrate dashboard literacy and reproducible research. Next, complete clean broker/data diagnostics and shadow sessions. Then, if justified, perform one bounded paper pilot. Finally, compare actual execution behavior with the documented assumptions. Progress only when the specific questions for the current phase have been answered.

Useful questions before expansion: Can I trace every fill to its authorization? Do actual exits match the journaled plan? Are costs complete? Do I understand unfilled and ambiguous outcomes? Is the data consistently usable? Have results been separated from operational errors? What evidence would make me stop?

Source basis: current controller, dashboard, studio, chat, scanner, rules, journal, exits, and backtest implementations under `src/alpaca_agents/`. See Volume 3 for the exact module map and external provider documentation. These worksheets are recommended operator practice, not additional application features or guarantees.

AlpacaAgent desk reference

Commands, controls, troubleshooting, and recovery

A practical reference for the current Windows implementation, with explicit command effects and evidence-based incident procedures.

DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

1. Quick command chooser	3
2. Launch modes and controls	4
3. Research and scanner commands	5
4. Diagnose startup, dashboard, and chat problems	6
5. Diagnose reconciliation and data failures	7
6. Ambiguous submissions and intent recovery	8
7. Manual flatten and stopping safely	9
8. History gaps, storage, locks, and backups	10
9. File map, metrics, and glossary	11
10. Corrections to older notes and source map	12
11. External sources and final readiness card	13

1. Quick command chooser

Reviewed against repository revision 235b087 on September 18, 2026. Run commands from C:\Users\eveli\OneDrive\Desktop\AlpacaAgent unless you deliberately use another checkout. Examples use python; substitute your working Python 3.11+ invocation if necessary.

Intent	Command or entry point	Effect
Open saved dashboard	Open-Dashboard.cmd	Rebuilds local HTML; opens browser
Generate dashboard	python -m alpaca_agents.dashboard	Reads local records; writes HTML
Serve local workspace	python -m alpaca_agents.controller --serve	Starts server; no periodic cycles
Query broker account	python -m alpaca_agents.executor account	Broker GET and request trace
Reconcile state	python -m alpaca_agents.executor reconcile	Broker reads; imports/updates local evidence
Read request traces	python -m alpaca_agents.executor traces	Local trace-store inspection
Inspect live intents	python -m alpaca_agents.executor intents	Journal read plus broker account access
Run one controller cycle	python -m alpaca_agents.controller	No POST; can update local state
Run repeated cycles	controller --every 300	Requires full module prefix; no POST without --submit
Run shadow scan	scanner --session DATE	Requires full module prefix; no broker orders
Replay cached history	backtest --bars-file FILE --symbol QQQ	Requires full module prefix; local research

Short module names in the last three rows mean python -m alpaca_agents.<module>. Full copyable examples appear in the following chapters. "No POST" does not mean "no disk writes": diagnostics, activity imports, cycle reports, and some dry reservations change local evidence.

Most important operational distinctions

Reconciled does not mean authorized. Authorized does not mean submitted. Submitted does not mean filled. A fill does not mean a lifecycle is fully closed. A recently rendered page does not mean the broker was recently queried. A backtest result does not mean option P&L was simulated.

Exit statuses

executor reconcile returns 0 for reconciled state and 2 for unreconciled state; operational failures generally return 1. A single controller cycle returns 0 when reconciliation is okay and 2 when it is not, even if later stages need closer inspection. The terminal loop uses 3 for designated state incidents, and 0 for its ordinary market-closed termination. Always read the report; exit status alone does not describe every stage.

2. Launch modes and controls

Static, served, and scheduled

```
python -m alpaca_agents.dashboard
python -m alpaca_agents.controller --runtime runtime --serve
python -m alpaca_agents.controller --runtime runtime --every 300 --dashboard
python -m alpaca_agents.controller --runtime runtime --serve --every 300
```

These are alternatives, not four commands to run simultaneously. One controller owns a runtime directory. --every must be at least 60 seconds. --serve --submit requires --every because on-demand chat diagnostics are always dry. Scheduled work in a served process may submit if the process was separately launched with all trading gates; chat itself still cannot submit.

The control file

Exact trimmed text	New entries	Exit handling
ARMED_PAPER	Eligible if all other gates pass	Eligible if other conditions pass
EXITS_ONLY	Blocked	Eligible if other conditions pass
DISABLED or anything invalid	Blocked	Blocked

No mode creates a process, supplies credentials, guarantees a fill, cancels an existing order, or clears an incident. Automatic submission still requires --submit. The control file is reread during decisions; playbook launch configuration is loaded when the controller initializes.

```
# Choose ONE intentional mode; these overwrite the file.
@'
from pathlib import Path
Path('runtime/trading-control').write_text('EXITS_ONLY', encoding='utf-8')
'@ | python
```

For a complete stop, substitute DISABLED after considering any positions and outstanding orders. For an approved paper pilot, substitute ARMED_PAPER. Direct marker writes are appropriate for a stopped controller; an operational process should use an atomic replacement procedure to avoid a partially written file. Invalid or incomplete reads fail closed.

Approvals are separate

The marker runtime/playbooks/trend_directional.approved must contain APPROVED, and the launch must include --enable-playbook trend_directional. Repeat --enable-playbook for multiple reviewed playbooks. Do not enable multiple playbooks during your first experiment just because the syntax permits it.

If using --runtime elsewhere, keep every executor database and control path aligned. Executor commands do not share the controller's --runtime flag: they expose individual --fills-db, --orders-db, --history-db, --trace-db, and --control-file arguments. Mixing directories can create misleading or account-inconsistent evidence.

3. Research and scanner commands

Fetch historical bars and report

```
python -m alpaca_agents.backtest --symbol QQQ `
--start 2023-01-01 --end 2026-09-17 `
--save-bars runtime/bars-QQQ.json `
--playbooks trend oversold_bounce breakout `
--output runtime/bt-QQQ.json
```

Both dates are required for a network run. This is an example window, not a permanently current command. Use an end date whose bars are complete. The market-data key belongs in the process environment.

```
python -m alpaca_agents.backtest --symbol QQQ `
--bars-file runtime/bars-QQQ.json `
--playbooks trend --output runtime/bt-QQQ.json
```

The cached run does not refetch history. It overwrites that output with the selected playbook report, so archive any multi-playbook report you want to retain. The dashboard discovers bt-*.json; the generative symbol tool expects bt-<SYMBOL>.json. CLI default backtest-<SYMBOL>.json is a different filename.

Standalone shadow scan

```
python -m alpaca_agents.scanner --session 2026-09-17 `
--symbols SPY QQQ IWM DIA `
--output runtime/shadow-scan.json
```

Set --session to the last completed exchange session. The option quotes are current, not historical quotes from that date. Default symbols are SPY QQQ IWM; DIA is optional. --audit can override runtime/market-data.jsonl, but audit and output must not be the same file.

Read reports without changing trading permissions

```
Get-Content runtime\shadow-scan.json -Raw | ConvertFrom-Json |
  ConvertTo-Json -Depth 12
Get-Content runtime\bt-QQQ.json -Raw | ConvertFrom-Json |
  Select-Object symbol, first_bar, last_bar, options_pnl_modelled, summary
```

The second command is a compact summary; inspect the full summary/trades objects for detailed research. Preserve source bars, exact command, code revision, and report filename for each important comparison.

Do not confuse these names

Backtest trend maps conceptually to trend-based scanner playbooks; it does not model a call versus a debit spread. Backtest breakout is not the launch string breakout_continuation. The current executable single-leg choices are trend_directional and oversold_bounce, subject to actual candidate structures and all other checks. The existence of a playbook name is not a recommendation to enable it.

4. Diagnose startup, dashboard, and chat problems

"python is not recognized" or "No module named alpaca_agents"

For a missing command, verify the Python installation and PATH or use `py -3` if available. For a missing module, install the project into the same interpreter from the project root using the command below. Its final dot denotes the current directory.

```
python -m pip install -e .  
python -m alpaca_agents.controller --help
```

Dashboard looks stale

Static mode requires regenerating HTML after runtime records change. Served questions reread local records, but main panels need a page refresh. Check the latest cycle timestamp, not only the rendered time. A browser connection does not prove the scheduled worker is still cycling.

Chat says it is rule-based

That is expected in static mode and in served mode without a valid `ANTHROPIC_API_KEY`. Restart the server after changing its environment. A model/provider failure or exhausted budget can also cause fallback. Read the answer's source label. Nova is generative-only.

Chat cannot answer a detailed file question

The model has a small allowlist of tools, not arbitrary filesystem access. It can read defined projections of snapshots, backtests, and cycles, plus rule explanations and optional market/news data. Open the actual JSON if your analysis needs fields the tool omits. Repeatedly asking does not grant the missing tool.

Houston did not run a diagnostic

In served mode, select Houston and send exactly `reconcile`. The server trims whitespace and compares case-insensitively. In static mode it cannot run a diagnostic. A sentence such as "reconcile and place this trade" is not the special command and cannot place an order.

Local URL no longer works

Keep the owning terminal open and use the URL printed by the current launch. The port and session token are launch-specific. Restarting the server can invalidate an old tab. The server is loopback-only; exposing it to another machine is not a supported operator shortcut.

Budget unexpectedly consumed

The generative cap counts provider calls, including tool rounds, not just questions. Usage is held in memory and resets with the process. It is not a durable billing ceiling. Check the provider's usage controls if you need a lasting financial limit.

5. Diagnose reconciliation and data failures

Work from exact reasons

```
python -m alpaca_agents.executor reconcile
python -m alpaca_agents.executor traces
```

Record the timestamp, all reasons, and relevant request identifiers. Do not replace an exact error with an informal label in your incident notes. Several reasons can occur together; fixing one does not establish full reconciliation.

Reason or symptom	Interpretation	First action
MARKET_CLOSED / NOT_A_SESSION	Trading-session gate	Check session context; inspect other reasons
BROKER_CLOCK_SKEW / STALE_STATE	Time evidence too old or inconsistent	Check system time and request latency
ACCOUNT_*	Account flags, eligibility, or status failed	Inspect current paper-account settings
NOT_CASH_ACCOUNT	Cash/margin policy not satisfied	Review account type and acknowledgement policy
HISTORY_*	Coverage, recency, or anchoring problem	Preserve import evidence; inspect exact suffix
POSITION_MISMATCH	Broker and local inventory disagree	Compare fills, activities, and positions
UNKNOWN_OPEN_ORDER	Order lacks expected local provenance	Inspect broker client order ID and journal
CLAIMED_INTENT_WITHOUT_BROKER_RECORD	Claimed intent lacks matching broker record	Treat as an ambiguous submission incident
HTTP 403 from option snapshots	Request not authorized for that data	Check credentials and exact data entitlement
Quote freshness rejection	Quote too old or invalid for valuation time	Inspect timestamps and provider response

Broker readiness and data readiness are independent

A successful account GET does not prove settled cash. A reconciled broker state does not prove options-data access. Stock bars working does not prove option snapshots work. A valid option snapshot does not prove every contract has complete eligible Greeks, interest, prices, or timestamps.

Quote age is capped at 120 seconds in the current snapshot adapter. The risk-state freshness window is 60 seconds. History coverage has its own fifteen-minute lag allowance. These are different clocks. Do not fix one by changing the others.

What to preserve

Keep cycles.jsonl, notifications.jsonl, market-data.jsonl, api-requests.sqlite3, activities.sqlite3, fills.sqlite3, and orders.sqlite3 together. The source-specific logs explain different parts of an incident. Include a sanitized excerpt and relevant IDs when reporting a bug; exclude credentials, full environment dumps, and unnecessary account data.

6. Ambiguous submissions and intent recovery

Why an unknown result is special

The broker may have accepted an order even if the client lost the response. The journal intentionally keeps an unknown submission claimed. The reserved capacity remains relevant until reconciliation or a controlled operator action resolves the situation. An HTTP failure is not automatically evidence that no order exists.

Investigation sequence

1. Stop new entries using the appropriate control mode; consider held positions before disabling exits.
2. Read the latest cycle, notifications, intents, and broker request traces.
3. Identify the authorization_id and client_order_id; do not guess from a ticker alone.
4. Inspect the paper broker's order history for that exact order and any fills.
5. Run reconciliation and review whether terminal broker records can be resolved by a controller diagnostic.
6. Use release-intent only when its specific prerequisites are met and your operator note documents the verification.

```
python -m alpaca_agents.executor intents
python -m alpaca_agents.executor traces
python -m alpaca_agents.executor reconcile
```

The intents command also needs paper credentials because it opens the account-bound journal after fetching the broker account. traces is the local inspection command.

Controlled release

The following is syntax, not an instruction to release an unidentified intent. Replace the placeholder with the exact authorization ID and write a factual note.

```
python -m alpaca_agents.executor release-intent AUTHORIZATION_ID `
--note "Verified exact client order ID in paper broker history at TIME; findings recorded in incident notes"
```

The command performs its own broker lookup. If it finds a terminal order, it can resolve from that status. If the order is still live, it refuses release. If the journal already has a broker acknowledgement, absence from one lookup is not sufficient for release. For an eligible unacknowledged claim, a broker not-found trace and operator note support the audited release.

Never use these recovery shortcuts

Do not submit the same intended trade again to see whether it works. Do not delete the order database, manually change claimed to rejected, remove a broker ID, or clear a loss latch. These actions destroy the evidence needed to prevent duplicate exposure and reconcile later fills.

Completion criterion

The broker's order/fill state and the journal agree, the incident is documented, and a fresh cycle has no unexplained state reasons. If uncertainty remains, leave it unresolved and obtain a code-level investigation rather than inventing a clean state.

7. Manual flatten and stopping safely

What flatten actually does

The manual command finds held quantity in the local fill ledger, reads the broker mark, creates an audited exit through the existing journal, and prices a day-limit sell at 95% of the mark with a \$0.01 floor. It does not retrieve the automatic manager's fresh NBBO bid. It is not an instant liquidation guarantee.

The manual implementation does not perform the controller's complete reconciliation sequence before preparing the exit. Run diagnostics first and confirm the exact held contract, quantity, account, and outstanding orders. The journal still applies its exit validation and control checks.

Preview first

Use the exact OCC contract copied from verified position evidence. OCC_CONTRACT below is a placeholder, not an instrument to trade.

```
python -m alpaca_agents.executor flatten OCC_CONTRACT
```

A successful dry preview prepares and then releases an unsent exit in the journal. It writes audit evidence; it is not a purely read-only display. Inspect contract, side, quantity, and limit. If the command says it cannot price or the ledger does not hold the contract, investigate rather than forcing the order.

Submit only when you intend the paper exit

```
python -m alpaca_agents.executor flatten OCC_CONTRACT --submit
```

Verify the returned outcome at the broker and reconcile after any fills. Do not rerun it after an ambiguous result. The command does not implement a general cancel/replace workflow for an existing unfilled order.

Pausing entries versus stopping all automation

For winding down, set EXITS_ONLY and keep a correctly configured submission-enabled controller running. For a complete automation stop, use DISABLED only after deciding how any open positions and pending orders will be handled. Both settings affect future local decisions; neither cancels broker orders already accepted.

If manual broker action becomes necessary to manage a position, preserve exact order/fill evidence and expect reconciliation to require attention. The normal workflow relies on orders originating in the local journal; unexplained external orders violate that provenance assumption.

Before closing the terminal

Confirm: positions flat or explicitly managed; pending broker orders understood; ambiguous intents resolved or explicitly escalated; runtime evidence preserved; next session's restart plan known. Do not use "dashboard shows no new notification" as evidence that the broker is flat.

8. History gaps, storage, locks, and backups

Activity imports require explicit bounds

Older notes show import-activities without its required arguments. The CLI requires --after and --until, using timezone-aware timestamps. Select bounds from the diagnosed gap and account history; do not copy a random range and assume completeness.

```
# Example syntax only: replace these bounds with the diagnosed window.
python -m alpaca_agents.executor import-activities `
  --after 2026-09-16T00:00:00+00:00 `
  --until 2026-09-18T12:00:00+00:00 --max-pages 100
python -m alpaca_agents.executor reconcile
```

Exhausting pages is not by itself verified history completeness. The reconciler requires anchored, continuous coverage from account creation and sufficient recency. Provider time bounds are exclusive, so merely touching import windows can leave a gap; use the code's overlap-aware coverage model. Never fabricate earlier fills to make inventory match.

Stale controller lock

```
Get-Content runtime\controller.lock
```

The file records a process ID and timestamp. Confirm that the specific old controller has terminated, including checking whether an ID has been reused for another process. Only after that verification, remove that exact stale file:

```
Remove-Item -LiteralPath .\runtime\controller.lock
```

Do not remove it to make room for a second running controller. A clean normal shutdown should remove the lock itself. The lock protects controller ownership, not every possible separate executor CLI invocation; avoid concurrent manual recovery commands while scheduled work is active.

Consistent backups

Stop active writers cleanly before making a filesystem backup of runtime. Preserve the directory as a unit, including any SQLite companion files present, reports, controls, approvals, and logs. Keep backups access-restricted because they contain account activity and operational evidence. The location under OneDrive does not make a concurrent copy a verified consistent SQLite backup.

Never restore an old runtime over active trading state simply to roll back an error. Broker orders and fills do not roll back with local files. A restored copy must be reconciled against current broker history before operation resumes. Keep one account per ledger/journal set.

Storage failures

If audit or persistence fails, the application stops the affected operation. Check disk space, permissions, locking, and filesystem availability. Preserve the error context. Clearing files to gain a clean dashboard can erase the very evidence needed to know whether an order was sent.

9. File map, metrics, and glossary

Runtime item	Purpose
trading-control	Exact local entry/exit mode
paper-margin-acknowledged	Explicit margin-paper policy acknowledgement
playbooks/*.approved	Reviewed playbook markers
controller.lock	Exclusive controller ownership marker
cycles.jsonl	One structured record per controller cycle
notifications.jsonl	Local warnings, incidents, and informational events
market-data.jsonl	Market-data request and filtering diagnostics
shadow-scan.json	Most recent standalone shadow report
bt-*.json	Dashboard-discovered underlying research reports
bars-*.json	Cached historical bars for repeatable research
api-requests.sqlite3	Broker request metadata and request identifiers
activities.sqlite3	Staged raw broker activities and import coverage
fills.sqlite3	Fill-level FIFO inventory, realizations, and loss latches
orders.sqlite3	Intent journal, stored bodies, and transitions
dashboard.html	Generated view; not the trading state authority

The separate TradeLedger/trades.sqlite3 prototype is not a second P&L source to add to FillLedger. Summing both can double-count realizations. Use the active fill-based accounting and understand its source coverage.

Key definitions

Basis: remaining acquisition cost, including allocated entry fees in the ledger. It differs from market value. **Mark:** broker-provided option price used as an observation, not a guaranteed executable price. **Bid/ask:** quoted buy/sell sides of the market; a limit still requires execution.

Reconciliation: comparison of broker account, positions, orders, clock, activities, and local evidence. **Reservation:** temporary capacity allocation. **Claim:** journal transition binding an intent to the prepared submission path. **client_order_id:** application-generated identifier used to correlate the broker order. **Request ID:** broker response identifier useful for support; it is not an order ID.

Mean R: average underlying outcome relative to modeled stop distance. **Median R:** middle outcome. **Profit factor:** aggregate positive R relative to absolute negative R, subject to zero-loss edge cases. **Win rate:** fraction of resolved outcomes with positive R. **Target-hit rate:** frequency of the target exit, not the same statistic as win rate.

no_fill: modeled trigger never produced an entry. **no_trade:** the modeled fill was already beyond a permitted stop/target boundary. **unresolved:** the dataset ended without a completed outcome. A report's **sample_sufficient** flag is a minimum-count check, not a proof of an edge.

10. Corrections to older notes and source map

Implementation details this guide deliberately corrects

- A DISABLED controller does not run the entry scanner. Use the standalone scanner for disabled shadow work.
- Network backtests require both --start and --end.
- import-activities requires --after and --until.
- Manual flatten uses 95% of broker mark, while automatic exits prefer a fresh bid when available.
- A dry enabled controller cycle can create a reservation; it is not entirely read-only.
- Default backtest filenames do not match dashboard bt-*.json discovery.
- Generative chat now has optional stock snapshot/news tools; descriptions saying it can only read local records are incomplete.
- Current trend entry plans have a 21-DTE time stop, not the replay's fifteen-session holding limit.
- Circuit-breaker decisions use CIRCUIT_BREAKER in the rules layer; do not expect the informal label BREAKER_TRIPPED everywhere.
- Cycle entries are nested under stages.entries.entries, and the scan summary is stages.scan. A generic halt_reason field is not the current cycle schema.

Local source map

All following paths are under src/alpaca_agents/.

controller.py defines CLI flags, runtime locking, cycle sequencing, approvals, and loop halts. gateway.py defines control-file interpretation. rules.py defines pure entry checks. scanner/scan.py, signals.py, and contracts.py define ideas, playbooks, exits plans, and structure filters. marketdata/snapshot.py defines snapshot normalization and quote-age checks.

executor/reconcile.py defines account, clock, history, inventory, and local cash/capital constraints. executor/orders.py defines reservations, claims, stored order bodies, exit preparation, and TTL. executor/submit.py implements single-attempt submission. executor/exits.py defines automatic exit evaluation. executor/__main__.py defines manual diagnostics, release-intent, and flatten.

dashboard.py defines report discovery and display projections. dashboard_chat.py and assets/dashboard.js define deterministic chat and browser behavior. studio.py defines the loopback server and exact Houston diagnostic request. llm_chat.py defines optional provider configuration, tools, usage counting, and fallback. backtest/replay.py and __main__.py define replay assumptions and report fields.

README.md, PLAYBOOK.md, and HANDOVER.md provide rationale and history, but contain superseded details. When upgrading the repository, compare these source locations before reusing operational commands from this dated guide.

11. External sources and final readiness card

Provider documentation

External pages were checked during preparation on September 18, 2026. They describe provider capabilities, which may be broader than this application's implementation. Provider pricing, permissions, supported products, and account rules can change; verify them before making account or subscription decisions.

[Alpaca paper-trading documentation](#) explains the simulated paper environment and its limitations. Paper execution is not equivalent to a demonstrated live execution record. The application pins its own paper endpoint; generic provider instructions for changing a base URL do not create a supported live configuration here.

[Alpaca options-trading documentation](#) describes brokerage options capabilities and order concepts. The broker may offer structures that this local journal refuses. Broker capability is not application implementation.

[Massive options API overview](#) describes option snapshots and market-data fields. [Massive options pricing](#) is the place to verify current subscription offerings. Confirm exact endpoint, quotes, and realtime access instead of relying on historical plan names in local notes.

This guide documents the code's internal risk and same-session policies. It does not assert that these implement every current legal, regulatory, or broker requirement. No provider fee schedule or subscription price is hardcoded into the recommendations here.

Before enabling a paper session

1. Correct runtime and paper account; current protected credentials.
2. Broker reconciliation understood and current for the session.
3. Required stock and options data available with eligible freshness.
4. One reviewed executable playbook and explicit symbol universe.
5. Matching approval marker and launch flag.
6. Intentional control mode and submission flag.
7. Known open positions and outstanding orders; no unexplained intent.
8. Machine and controller will remain available for the planned period.
9. Exit behavior and known research/execution gaps understood.
10. Session plan, incident procedure, and review record ready.

If anything looks wrong

Read the exact record, preserve the evidence, and identify the earliest failed condition. Pause new risk when appropriate. Keep existing positions explicitly managed. Do not clear databases, loosen checks, retry unknown submissions, or mistake a missing record for a zero value. Resume only after the relevant broker and local evidence agree.